Section 14.5: Social & Discovery Platforms

> *Part 5 of 8 in the System Design Interview Series*

Executive Overview

Social platforms handle graph-structured data, real-time notifications, and personalised recommendations. The core interview challenges are: graph query efficiency at scale, news feed fan-out (covered in 14.1 from a different angle here), recommendation system architecture, and notification delivery pipelines. Understanding approximate algorithms (MinHash, Count-Min Sketch, Bloom filters) is what separates strong candidates at senior levels.

---

1. Design a Location-Based Service (Yelp / Google Maps)

Requirements Analysis

Dimension	Requirement
Functional	Search businesses, show on map, directions, reviews
Scale	100M businesses, 1B location queries/day
Latency	< 100ms for nearby search
Data	Geospatial + attribute filtering combined

Spatial Indexing Comparison

Index	Update Speed	Radius Query	Polygon Query	Memory	Best For
Geohash	O(1)	O(K cells) — fast	Poor	Low	Distributed, Redis-compatible
S2 Geometry	O(1)	O(K cells) — fast	Excellent	Low	Complex region queries

Index	Update Speed	Radius Query	Polygon Query	Memory	Best For
R-tree (PostGIS)	O(log N)	O(log N + K) — medium	Excellent	Higher	SQL-based, complex shapes
QuadTree	O(log N)	O(log N + K)	Good	Medium	Hierarchical, adaptive density

Geohash mechanics:

```
Location: (37.7749, -122.4194) — San Francisco

Encoding at 6 characters: "9q8yy9" → ~1.2km × 0.6km cell
Encoding at 8 characters: "9q8yy9nq" → ~38m × 19m cell

Nearby search at 5km radius:
→ Expand geohash to contain the radius
→ Query current cell + 8 neighbouring cells (3×3 grid)
→ Filter results within exact radius in application layer
```

Why 9 cells, not just the current cell? A business at the edge of your geohash cell might be closer than one in the centre of an adjacent cell. The 3×3 grid guarantees no nearby business is missed regardless of where within your cell you are.

Query Pipeline

```
User: "Restaurants near me, price ≤ $$, rating ≥ 4.0"

1. Geocode user location → geohash
2. Fetch business IDs from spatial index (geohash lookup)
3. Filter by category (restaurants), price, rating — Elasticsearch
4. Rank: distance × 0.4 + rating × 0.3 + review_count_log × 0.2 + popularity × 0.1
5. Return top 20 results with distance, thumbnail, hours
```

Elasticsearch for filtering: Geospatial + attribute filtering is a natural fit. ES supports `geo_distance` queries with additional `term` filters for category/cuisine, `range` for price/rating. Combined query executes in < 50ms at 100M document scale.

Interview Problem 1 — Google Maps "Find Nearby" at Scale

"Design the backend for Google Maps 'find nearby restaurants' that handles 1B location queries/day with < 100ms P99 latency."

Solution framework:

1. **Write path**: Businesses write location + metadata to PostgreSQL (source of truth). A CDC (change data capture) pipeline propagates updates to Elasticsearch within 30 seconds.
2. **Read path**: Queries hit Elasticsearch directly (not Postgres). ES cluster is sized for the full 100M business dataset with 30 replicas for read throughput.
3. **Caching layer**: Cache results for popular (lat/long, radius, filters) combinations in Redis with 5-minute TTL. Geohash the query location to group similar queries. ~70% of queries are for popular areas.
4. **Personalisation**: Post-processing layer re-ranks Elasticsearch results using user preferences (cuisine history, price preference) stored in user profile service.
5. **Capacity math**: 1B queries/day = 11,600 QPS average. With Redis cache at 70% hit rate, Elasticsearch handles ~3,500 QPS. Each shard handles ~500 QPS → ~7 primary shards needed.

---

2. Design a Social Network (Facebook / Instagram)

Requirements Analysis

Dimension	Requirement
Functional	Connections, news feed, posts, comments, likes
Scale	3B users, 1B DAU, 50M posts/hour
News Feed	Personalised ranking per user

Graph Data Model

```
-- Adjacency list representation
CREATE TABLE friendships (
    user_a    BIGINT NOT NULL,
    user_b    BIGINT NOT NULL,
    created_at TIMESTAMP,
    PRIMARY KEY (user_a, user_b)
);
CREATE INDEX idx_user_b ON friendships(user_b);  -- Bidirectional lookup
```

```
-- Partitioning: shard friendships by user_a % num_shards
-- Cross-shard queries (mutual friends) require scatter-gather
```

Graph database alternative (Neo4j): - Better for complex traversals (friends of friends of friends) - Less suited for 3B user scale - Most social networks use sharded adjacency lists, not graph DBs

News Feed Architecture

Same hybrid fan-out principle as 14.1 but with ranking:

```
Post created → Fan-out to followers' timeline caches (normal users)
             → Stored centrally (celebrities)

Timeline read:
  1. Fetch pre-cached posts (last 1,000 from followed accounts)
  2. Merge in celebrity posts for accounts user follows
  3. Rank using ML model (EdgeRank-style)
  4. Return top 50 posts
```

EdgeRank-style scoring:

```
post_score = affinity(viewer, poster) × content_weight × time_decay

Where:
  affinity   = normalised interaction history (likes, comments, DMs)
  weight     = type multiplier (video > photo > link > text)
  time_decay = e^(-λ × hours_since_post), λ tuned per content type
```

Privacy Controls Implementation

```
Post visibility: PUBLIC | FRIENDS | CUSTOM | ONLY_ME

At query time:
SELECT posts WHERE poster_id IN (followed_users)
  AND visibility != 'ONLY_ME'
  AND (visibility = 'PUBLIC'
    OR (visibility = 'FRIENDS' AND friendship_exists(viewer, poster))
    OR (visibility = 'CUSTOM' AND viewer IN post.allowed_viewers))
  AND viewer NOT IN poster.blocked_users
```

Performance concern: Blocked user check on every post fetch is expensive. Solution: Cache blocked users list in viewer's session (small list, rarely changes).

Interview Problem 2 — Mutual Friends at Scale

"Design the 'X mutual friends' feature for a platform with 1B users. Two users may each have up to 5,000 friends. How do you efficiently compute and display mutual friends?"

Solution framework:

1. **Naive approach**: Load friend list A (5K users) and friend list B (5K users), compute set intersection. O(min(|A|, |B|)) = fast enough for individual queries but doesn't scale if done live for every profile view.
2. **Pre-computation**: Run a background job every 6 hours that computes mutual friends for all user pairs who have viewed each other's profile recently. Store in Redis: `mutuals:{user_a}:{user_b} → [user_ids, count]` with 24h TTL.
3. **Cache-first flow**: Profile view triggers Redis lookup. Cache hit → immediate return. Cache miss → compute inline (5K intersection is < 1ms in application memory), store result, return.
4. **Approximate count (for display only)**: Use MinHash to estimate Jaccard similarity between two friend sets. 128 hash functions → ±3% error. Store 128-byte MinHash signature per user. Dot product gives approximate mutual count in O(128) operations.
5. **Bloom filter optimisation**: For the "any mutual friends?" binary check, store each user's friend list in a Bloom filter (false positive rate 1%). Intersection of two Bloom filters quickly tells you if mutual friends likely exist before expensive exact computation.

---

3. Design a Recommendation System (Spotify / Netflix)

Requirements Analysis

Dimension	Requirement
Functional	"Recommended for you" personalisation
Scale	100M users, 10M items
Latency	< 100ms for recommendations
Freshness	New recommendations daily

Two-Stage Architecture

All production recommendation systems use this pattern:

```
Stage 1 — Candidate Generation (billions → thousands):
  Purpose: Fast retrieval of potentially relevant items
  Methods: Collaborative filtering, content similarity, trending
  Output: ~1,000–10,000 candidates per user

Stage 2 — Ranking (thousands → tens):
  Purpose: Precise scoring with rich features
  Methods: Deep neural network, gradient boosted trees
  Output: Top 20–50 items for display
```

Why two stages? Stage 1 must be fast (milliseconds) and cheap (approximate). Stage 2 can be expensive (heavy ML model) but only runs on a small candidate set. Running the ranking model on all 10M items per user would be impossible.

Collaborative Filtering

User-item matrix:

```
         Song A  Song B  Song C  Song D
User X:  1       0       1       0       (listened)
User Y:  1       1       0       0
User Z:  0       0       1       1

Similarity(X, Y) = Cosine similarity of their vectors = 0.5
Similarity(X, Z) = 0.5

Recommendation for X:
  - Y liked Song B (X hasn't heard it) → recommend Song B
  - Z liked Song D (X hasn't heard it) → recommend Song D
  - Weight by similarity score
```

Matrix factorisation (production approach): - Decompose user-item matrix into user embeddings (U) and item embeddings (I) - Each user/item is a 128-dimensional vector - `score(user, item) = U[user] · I[item]` (dot product) - Train offline on Hadoop/Spark; refresh daily - At serve time: ANN (approximate nearest neighbour) search over item embeddings

Candidate Generation Methods Comparison

Method	Coverage	Novelty	Computation	Handles Cold Start?
Collaborative filtering	Good (popular items)	Low (echo chamber risk)	High (offline)	No
Content-based	Limited (user history biased)	High	Low	Yes

Method	Coverage	Novelty	Computation	Handles Cold Start?
Trending/ Popular	High	Low	Very low	Yes
Hybrid	Excellent	Medium	Medium	Partial

Cold start problem: New users have no history. Solutions: Ask for explicit preferences onboarding; use demographic signals; start with global popular items; rapidly incorporate first few interactions.

Interview Problem 3 — Real-Time Recommendation Updates

"A user just listened to a new artist for the first time. How quickly do their recommendations update to reflect this, and how do you implement this?"

Solution framework:

1. **Event capture**: Listening event emitted to Kafka within 1 second of action.
2. **Real-time feature store update**: Stream processor (Flink) consumes Kafka events and updates the user's recent-activity feature vector in Redis: `user:features:{user_id} → {recent_artists, recent_genres, session_context}`. Updated in < 5 seconds.
3. **Session-level recommendations**: The next recommendation request reads the updated feature store and passes context features to the ranking model. Recommendations reflect the new artist within 1–2 request cycles (~30 seconds in practice).
4. **Batch model retraining**: The deeper collaborative filtering model (user embedding) is retrained daily on Spark. New artist preference fully incorporated by next day.
5. **Trade-off articulated**: Real-time features (< 30s) handle session context. Batch model (24h) handles deep preference shifts. This two-tier freshness model balances latency and model quality.

---

4. Design a Notification Fan-Out Platform

Requirements Analysis

Dimension	Requirement
Functional	Push, email, SMS, in-app notifications

Dimension	Requirement
Scale	100M users, 1B notifications/day
Delivery	Real-time or batched based on priority

Fan-Out Strategy Comparison

Strategy	Delivery Speed	Scale Efficiency	Celebrity Problem	Complexity
Fan-out-on-write	Immediate	Poor (O(followers) per event)	Very poor	Low
Fan-out-on-read	On open (delayed)	Excellent	Excellent	Medium
Priority-tiered queue	Immediate for critical	Good	Good	High

Tiered queue model:

```
Event arrives → Classify priority:
  CRITICAL (DM, mention, security): → Immediate fan-out → push within 5s
  STANDARD (like, comment, follow): → Batch within 60s → push notification
  DIGEST   (weekly summary):        → Aggregate → email once daily

Per-user quiet hours:
  Suppress STANDARD notifications 10pm–8am in user's timezone
  CRITICAL always delivered
```

Delivery Pipeline

```
Notification Service → Channel Router
                              ↓
        ┌─────────────────────┼──────────────────────┐
        ↓                     ↓                      ↓
   Push (APNs/FCM)    Email (SendGrid/SES)    SMS (Twilio)
        ↓                     ↓                      ↓
   Delivery receipt    Open tracking          Delivery status
        ↓                     ↓                      ↓
                  Unified delivery log (Cassandra)
```

Rate limiting per channel: Prevent spam. Max 10 push notifications/hour per user from a single app. Max 5 emails/day. Users can customise in preferences.

Section Summary: Social & Discovery Patterns

Challenge	Recommended Approach	Key Trade-off
Nearby search	Geohash + Elasticsearch	Geohash boundary edge cases
Graph mutual friends	Pre-computed cache + MinHash approx	Staleness vs. computation cost
News feed ranking	Two-stage: fan-out + ML rank	Write amplification for large accounts
Recommendations	Two-stage: candidate gen + neural rank	Freshness vs. model quality
Notifications	Tiered priority queue	Complexity vs. delivery speed control